

دليل المطور الشامل

التعامل مع الملفات في بايثون

تعلم كيفية قراءة وكتابة وإدارة البيانات بشكل دائم باستخدام لغة البرمجة الأكثر مرونة في العالم.



الأساسيات والمفاهيم

فهم الفرق بين الذاكرة المؤقتة والتخزين الدائم

لماذا نحتاج للملفات؟



مشاركة المعلومات

تسمح الملفات للبرامج المختلفة بتبادل البيانات، مثل ملفات CSV أو JSON أو السجلات (Logs).



استدامة البيانات

تُفقد المتغيرات عند إغلاق البرنامج. الملفات تضمن بقاء البيانات للأبد على القرص الصلب.

أوضاع فتح الملفات (Modes)



الإلحاق 'a'

يضيف البيانات في نهاية الملف دون
مسح المحتوى القديم.



الكتابة 'w'

ينشئ ملفاً جديداً أو يمسح
محتويات الملف الحالي ليكتب فوقه.



القراءة 'r'

للوصول للبيانات فقط. يفشل إذا
لم يكن الملف موجوداً.

Opening
files

01

Reading
from files

02

Writing
to files

03

Adding
to files

04

Closing
the files

05

File handling in Python

تدفق العمليات

العملية البرمجية تتبع تسلسلاً منطقياً يضمن سلامة البيانات وموارد النظام.

✓ تحديد المسار والوضع المناسب

✓ طلب الوصول من نظام التشغيل

✓ إجراء العمليات (قراءة/كتابة)

✓ إغلاق الاتصال لتحرير الذاكرة

استراتيجيات قراءة البيانات

`()read`

تقرأ كامل محتوى الملف ككتلة واحدة (String). مفيدة للملفات الصغيرة.

`()readline`

تقرأ سطرًا واحدًا في كل مرة. مثالية لمعالجة الملفات ضخمة الحجم لتقليل استهلاك الذاكرة.

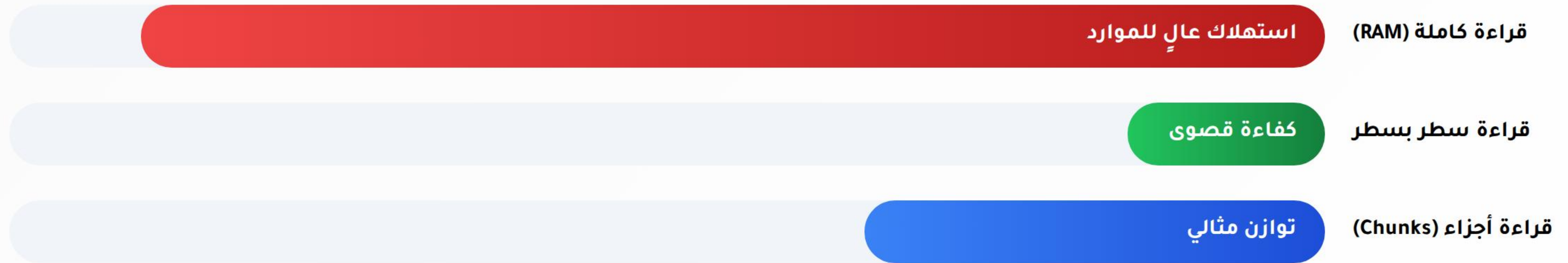
`()readlines`

تقرأ جميع الأسطر وتضعها في قائمة (List) حيث كل عنصر يمثل سطرًا.

مقارنة أوضاع التعديل

الوضع	وجود الملف	محتوى الملف القديم	موضع المؤشر
'w' (Write)	ينشئ ملفاً جديداً	يتم مسحه بالكامل	البداية
'a' (Append)	ينشئ ملفاً جديداً	يتم الحفاظ عليه	النهاية
'x' (Create)	يعطي خطأ (Error)	N/A	البداية

كفاءة القراءة في الملفات الضخمة



يُنصح باستخدام الحلقات التكرارية (Loops) للملفات التي تتجاوز حجم ذاكرة الوصول العشوائي المتوفرة.

أفضل الممارسات: استخدام `with`



إدارة السياق (Context Manager)

الطريقة التقليدية تتطلب إغلاق الملف يدوياً باستخدام `file.close()`، وهو أمر قد يُنسى ويسبب تسرباً للموارد.

```
with open('file.txt', 'r') as f:  
    data = f.read()  
# الملف يُغلق تلقائياً هنا
```


التعامل الآمن مع الأخطاء

تجنب انهيار البرنامج

دائماً توقع عدم وجود الملف أو عدم توفر الصلاحيات الكافية. استخدم كتل `try...except` للتعامل مع هذه الحالات بذكاء.

التحقق من مسار الملف قبل البدء. 

التقاط خطأ `FileNotFoundError`. 

تنظيف الموارد في كتلة `finally`. 



دورة حياة معالجة الملف

1. الفتح

استخدام `open()` مع
تحديد الوضع والترميز
(Encoding).

2. المعالجة

قراءة المحتوى أو كتابة
سجلات جديدة في الملف.

3. التأكد

التحقق من اكتمال العمليات
بنجاح ودون أخطاء.

4. الإغلاق

تحرير مقبض الملف (File
Handle) ليعود لنظام
التشغيل.

شكراً لكم!

هل لديكم أي أسئلة حول التعامل مع الملفات؟



www.python.org



Python Dev Team

Image Sources

https://miro.medium.com/1*siXqQ6ZQsgo9yN_zMxboh.png

Source: iamdamilare13.medium.com



<https://k21academy.com/wp-content/uploads/2021/10/zed-1024x640.jpg>

Source: k21academy.com



https://png.pngtree.com/png-vector/20260330/ourlarge/pngtree-data-synchronization-icon-with-document-and-checkmark-vector-png-image_19022311.webp

Source: pngtree.com

